

University of Eswatini

Department of Computer Science

CSC411: Integrative Programming Technologies

Assignment 1: Mini Project

Automating an Inter-Organizational Process using REST-Based Integrated Systems

Process: Justice Administration

Team Members

#	Name	Student No.
1	Ndlovu Dumisa	202406255
2	Mkhatshwa Fisokuhle	202404184
3	Ginindza Nkosephayo	202202143
4	Khumalo Owenkhosi	202200280
5	Magagula Sibonelo	202203691

Programme: BSc IT | Instructor: Dr MA Nxumalo

Due: 4 November 2026, 23h00 (Moodle)

*GitHub Repository:*https://github.com/Phayo121/justiceflow

1. Introduction

1.1 Background

Justice administration in Eswatini involves several institutions that each hold part of a person's case. The police investigate and arrest, prosecution authorities decide whether a matter proceeds to court, the courts hear the matter, and Correctional Services hold people who are remanded or sentenced. A case therefore passes through at least four organizations before it ends in closure, release or completed custody.

The process assigned to our group is Justice Administration. The course brief describes it as a tool that processes a suspect when handed to the police, through the administration of justice in court, up to custody at the correctional services. This report documents JusticeFlow, a prototype that supports that chain of hand-overs through a shared case record and a separate, role-restricted workspace for each organization.

1.2 Problem Statement

Where each agency keeps its own files, case details are re-typed or physically carried at every hand-over. This causes delays, transcription errors and lost files. No agency can easily tell which stage a case has reached, and there is no single history of who acted on the case and when. Access is also hard to control: staff may see more than their role requires, and records can change without a trace. The time a person spends waiting between stages can depend on the slowest hand-over in the chain.

1.3 Aim and Objectives

Aim: to design and implement a prototype that moves a case between Police, Prosecution, Court and Corrections through one authenticated, role-controlled system.

Objectives:

1. To describe the inter-organizational justice process, its stakeholders and its data flows (Section 2).
2. To design an integrated, REST-style architecture with role-based access control and an audit trail (Section 3).
3. To implement a working demo in Node.js with persistent SQLite storage and a web workspace for each agency (Section 4).
4. To enforce access rules and case-stage transitions on the server, not only in the interface.
5. To test the system and state its limitations honestly.

1.4 Scope and Report Structure

In scope: the hand-over of a case from police intake to prosecution, court and custody; staff sign-in and role-based access; case activity and audit records; and a public landing page with a rights overview that links to Chapter III (Fundamental Rights and Freedoms) of the Constitution. Out of scope: deciding arrest, charge, guilt, bail, sentence or release, which remain decisions

for the responsible officials; evidence management; integration with live agency systems; and any use of real case data.

The rest of the report is organized as follows. Section 2 analyses the process and its stakeholders. Section 3 gives the requirements and the system design. Section 4 describes the implementation and testing. Section 5 concludes and Section 6 lists the references.

2. Requirement Analysis

2.1 Process Description

The process starts when a suspect is handed to the police. A police officer creates the initial case record in the Police workspace. When the police have finished their part, the officer refers the case to Prosecution.

Prosecution staff see the referred case and record one of two outcomes: a referral to court, or an administrative closure. Court staff then see the court-stage case and record either a custody handoff or an administrative closure. When a case is handed to custody, Correctional Services staff see it in their workspace and can record an administrative closure.

At every step the system records who acted and when. Only the agency that currently holds a case can move it forward, and the server checks each move. The system records stages and hand-overs. The decisions themselves, such as charging, bail, verdict, sentence or release, are made by the responsible officials outside the system.

2.2 Stakeholders and Roles

Organization / Actor	Role in the process	Data it owns	Data it needs
Police (Royal Eswatini Police Service)	Receives the suspect, creates the initial case record and refers it to Prosecution.	Initial case record and police-intake details.	Suspect and allegation details from the arrest.
Prosecution	Reviews referred cases and records a court referral or an administrative closure.	The prosecution decision on each case.	The case record referred by Police.
Court (Judiciary)	Handles court-stage cases and records a custody handoff or an administrative closure.	The court-stage record and its decision.	The case record referred by Prosecution.
Correctional Services	Holds cases handed to custody and records an administrative closure.	The custody-stage record and its closure.	The case record handed off by the Court.
System administrator	Creates staff accounts and supplies temporary passwords. Cannot view case records.	User accounts.	None. Has no access to case data.
Public / accused person	Not a system user. Can read the public landing page and rights overview.	None in the system.	Rights information on the landing page.

2.3 Activities and Flow

1. The administrator creates Police, Prosecution, Court and Corrections accounts and gives each user a temporary password. Users change it before opening a workspace.
2. Police staff sign in and create the initial case record. The case is now in the police-intake stage.
3. Police refer the case to Prosecution. The case moves to the prosecution stage and leaves the Police workspace.
4. Prosecution staff review the case and record a court referral or an administrative closure.
5. Court staff see the court-stage case and record a custody handoff or an administrative closure.
6. Correctional Services staff see the custody case and record an administrative closure when the matter ends.
7. At every step the system writes a case-activity entry and an audit event.
8. Exceptions: a move requested by the wrong role, or from the wrong stage, is rejected by the server. Repeated failed sign-ins are throttled.

2.4 Data Flow Diagram

Figure 2.1: Level 0 (context) data flow diagram. Police, Prosecution, Court, Corrections and the administrator exchange data with one system, JusticeFlow.

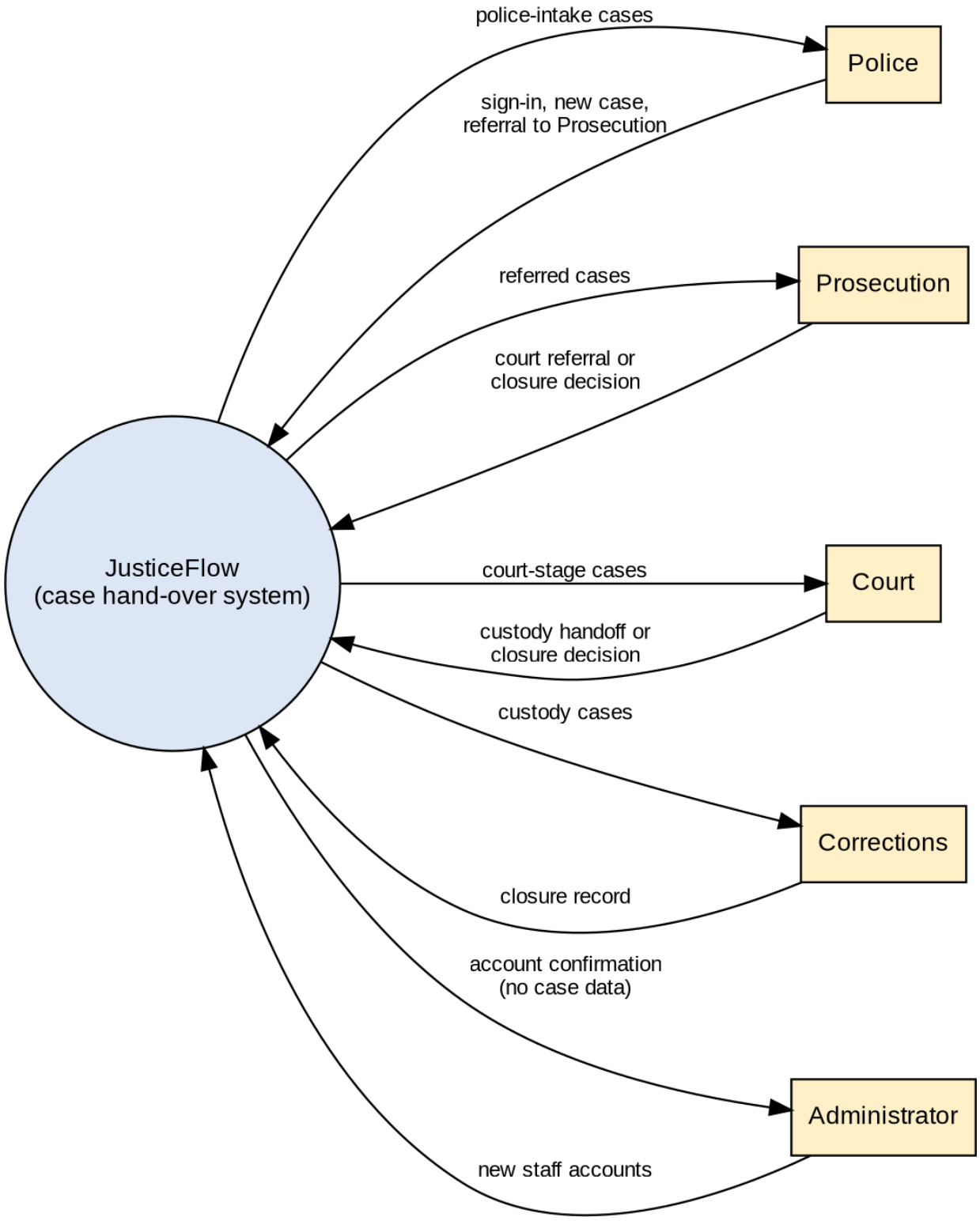
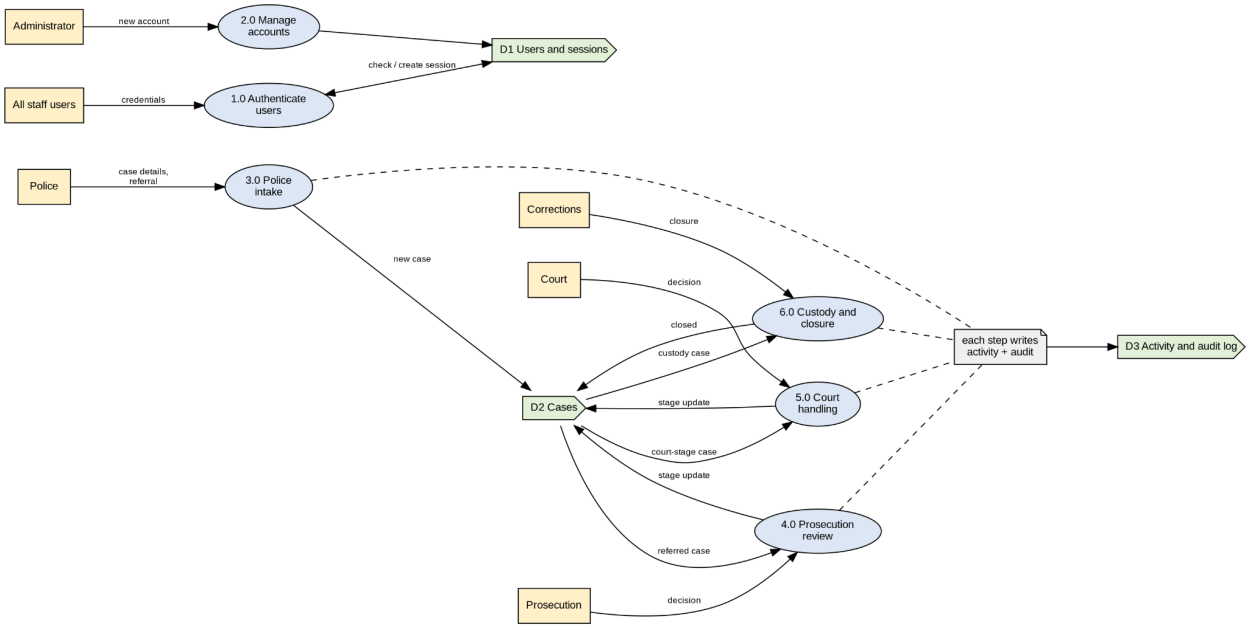


Figure 2.2: Level 1 data flow diagram. Processes: authenticate, manage accounts, police intake, prosecution review, court handling and custody. Data stores: users and sessions, cases, activity and audit log.



2.5 Current Problems and Opportunities for Automation

- Repeated data entry: each agency re-captures the same case details. One shared record, entered once, removes this.
- Unclear status: without a shared record no agency can say which stage a case has reached. A stage that changes only through controlled hand-overs gives one answer.
- Weak accountability: paper files do not show who changed what. Activity and audit entries record the actor and time of each change.
- Uncontrolled access: staff may see cases outside their role. Server-side role and stage checks limit each agency to its current-stage records.
- Delay at hand-overs: a case can wait in a tray or inbox. An electronic hand-over places it in the next agency's workspace at once.

3. System Design

3.1 Requirements

3.1.1 Functional Requirements

ID	Requirement	Service
FR1	Staff shall sign in with a username and password and sign out.	Authentication
FR2	The system shall have no public sign-up. Accounts are created only by an administrator.	Admin
FR3	The administrator shall create Police, Prosecution, Court and Corrections accounts with a unique username and temporary password. Users shall change the password before opening a workspace.	Admin
FR4	Police staff shall create an initial case record and view police-intake records.	Police workspace
FR5	Police staff shall refer a case to Prosecution.	Police to Prosecution handoff
FR6	Prosecution staff shall view referred cases and record a court referral or an administrative closure.	Prosecution workspace
FR7	Court staff shall view court-stage cases and record a custody handoff or an administrative closure.	Court workspace
FR8	Corrections staff shall view custody cases and record an administrative closure.	Corrections workspace
FR9	Each agency shall see only the cases in its current stage. A closed case shall remain visible only to the agency that closed it. The administrator shall not see case records.	Access control (all)
FR10	The system shall record case	Activity and audit log

ID	Requirement	Service
	activity and an audit event for every case write.	

3.1.2 Non-Functional Requirements

The prototype must also meet the following quality requirements, which apply to the whole system rather than to one function.

ID	Category	Requirement
NFR1	Security	Passwords are salted and hashed with scrypt. Session tokens are random, stored as hashes and sent in HttpOnly, SameSite cookies. Role and stage checks run on the server. Security headers are set and login attempts are throttled.
NFR2	Interoperability	All endpoints exchange JSON over HTTP using standard methods, so other tools or agency systems, and Postman, can call them.
NFR3	Performance	Normal case operations should respond without noticeable delay on a single machine. Request size limits protect the server.
NFR4	Reliability / Availability	Records persist in a SQLite database file and survive restarts. Production availability, backup and failover are outside the prototype.
NFR5	Auditability	Every case write creates case-activity and audit entries with the actor and time. The log is an application log, not a certified tamper-proof system.
NFR6	Data protection	The prototype must not be used with real names, allegations, evidence or custody data. Real use needs legal and data-protection validation.
NFR7	Usability	Each role has its own workspace page that shows only the actions available to that role.
NFR8	Maintainability / Portability	Runs on Node.js 22.13 or later

ID	Category	Requirement
		with no third-party packages. The database path is set with JUSTICE_DB_PATH. Checked with npm run check and npm test.

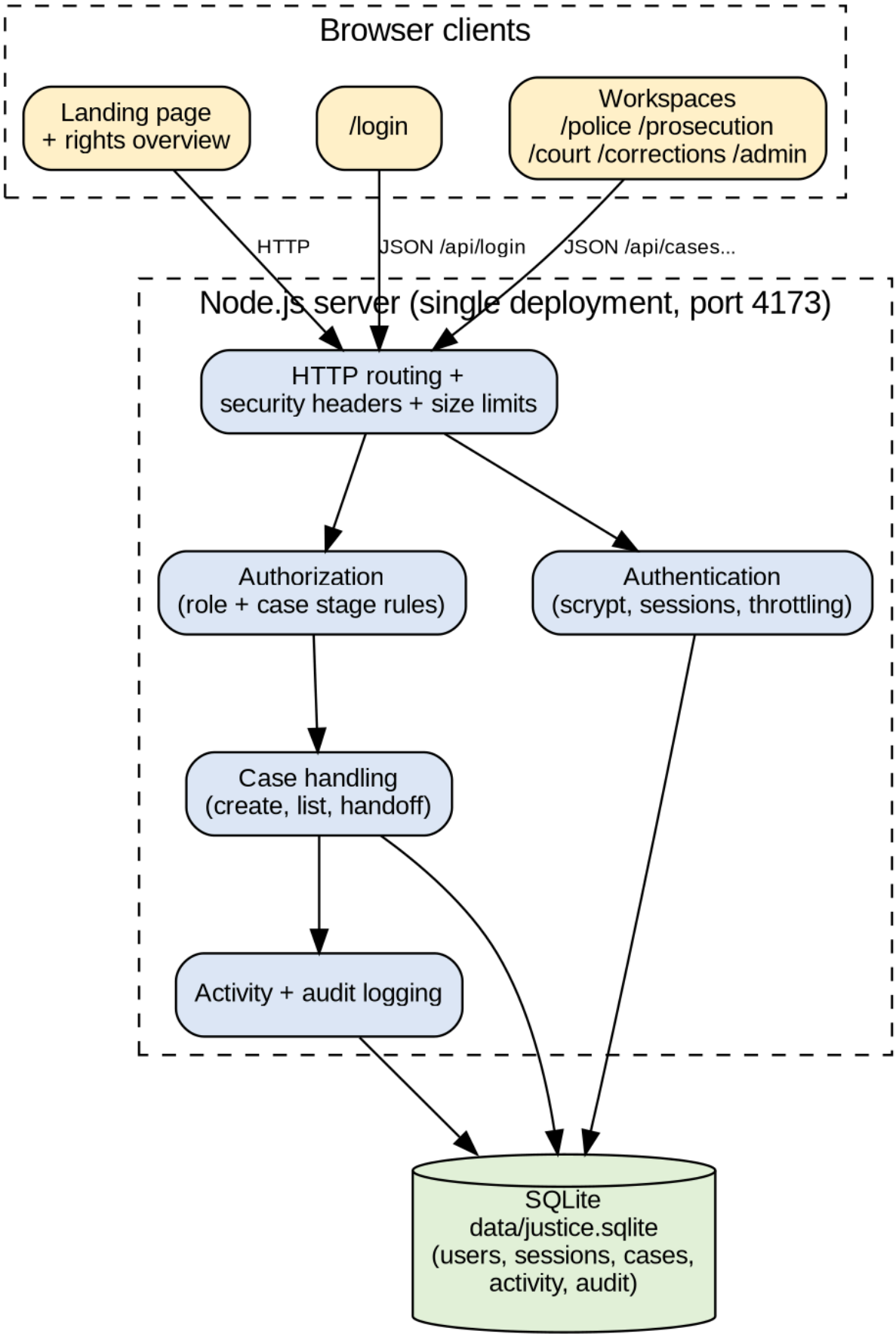
3.2 Design Decisions and Justification

Element	Our choice	Justification
Architecture style	REST-style JSON API with a role-based workspace for each agency, served by one Node.js server.	Keeps the demo easy to run and test while still separating each agency's actions, visible data and pages. The agencies are modules of one deployment, not independently deployed services (see Section 4.7).
Communication	HTTP with JSON bodies and cookie-based sessions, using request/response calls.	Standard and easy to test with Postman. Hand-overs are POST calls to /api/cases/:id/handoff.
Design patterns	Role-based access control, a case-stage state machine, server-side validation and an audit log.	Role checks limit each agency to its own actions. The stage rules let a case move only through valid hand-overs. The audit log gives accountability.
Backend language/framework	Node.js 22.13 or later using built-in modules only, with no third-party packages.	Fewer dependencies means simpler setup and less supply-chain risk. Built-in modules cover HTTP, hashing (scrypt) and SQLite.
Database	SQLite through node:sqlite, stored in data/justice.sqlite. Another file can be chosen with JUSTICE_DB_PATH.	Persistent, needs no database server and is one file to back up. Adequate for a prototype. A production system would need a managed database.
Client	Web pages served by the same server: a public landing page, /login and one workspace page each for /police, /prosecution, /court, /corrections and /admin.	Works in any browser with nothing to install. One page per role keeps each interface simple.
Tools	Git and GitHub, Postman or Insomnia, draw.io, and the npm	Version control for the team, API testing, diagrams and repeatable

Element	Our choice	Justification
	scripts check and test.	validation.

3.3 Architecture Diagram

Figure 3.1: Architecture of JusticeFlow. Browser pages for each role call the JSON API of the Node.js server. The server authenticates the session, checks the role and the case stage, and reads or writes the SQLite database.


Browser clients
Landing page
+ rights overview
/login
Workspaces
/police /prosecution
/court /corrections /admin
HTTP
JSON /api/login
JSON /api/cases...
Node.js server (single deployment, port 4173)
HTTP routing +
security headers + size limits
Authorization
(role + case stage rules)
Authentication
(scrypt, sessions, throttling)
Case handling
(create, list, handoff)
Activity + audit logging
SQLite
data/justice.sqlite
(users, sessions, cases,
activity, audit)

Components: browser pages (landing page, login and the five workspaces); HTTP server with routing and security headers; authentication and session module; authorization rules (role and stage); case handling; SQLite database holding accounts, sessions, cases, case activity and audit events.

3.4 REST API Specification

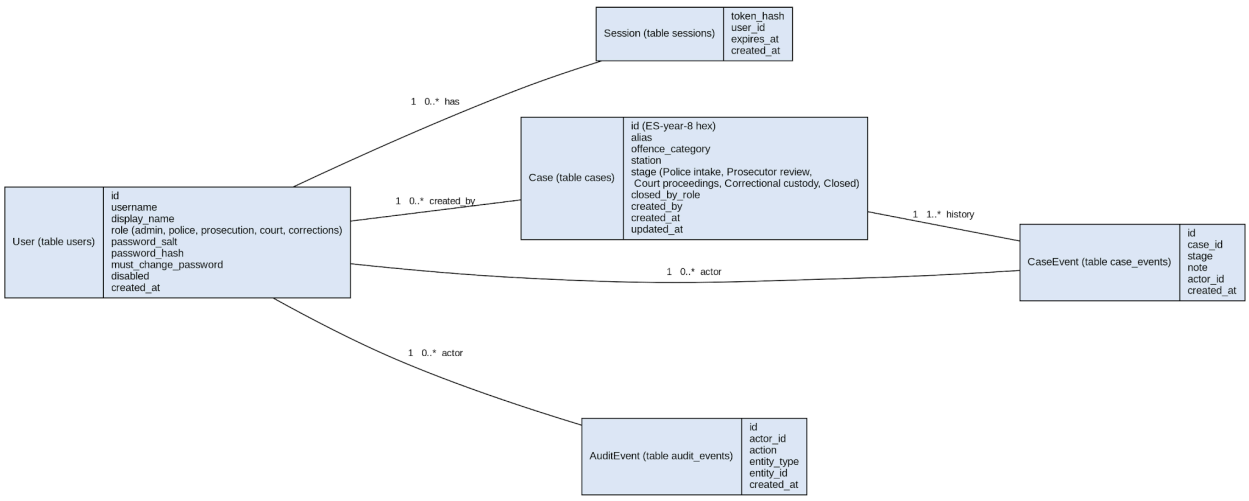
The API is grouped by purpose. The Purpose column says which role may call each endpoint. Requests that fail the role or stage checks are rejected by the server.
Status codes used: 401 not signed in, 403 role not allowed, 404 case not visible to the role, 409 invalid transition, 428 password change required, 429 too many sign-in attempts.

Method	Endpoint	Purpose	Request / Response
POST	/api/login	Sign in (public).	Body: username, password. Response: workspace path and mustChangePassword flag; sets the justice_session cookie. 401 for wrong details; 429 after 10 failed attempts in 15 minutes.
POST	/api/logout	Sign out (any signed-in user).	Response: ok. The session is deleted and the cookie cleared.
GET	/api/session	Return the current session (any user).	Response: username, display name and role. 401 if not signed in, 428 if a password change is pending.
POST	/api/password	Change password. Required after signing in with a temporary password.	Body: password (the new password, 12 to 128 characters). Response: workspace path. The user's other sessions are ended.
POST	/api/admin/users	Create a staff account (administrator only).	Body: username, displayName, role, password. Response: 201 with username and role. 409 if the username exists; 403 if not an administrator.
GET	/api/cases	List the cases the caller's role may see: current-stage cases, plus closed cases for the agency that closed	Response: 200 with up to 200 cases. 403 for administrators.

Method	Endpoint	Purpose	Request / Response
		them.	
POST	/api/cases	Create an initial case record (Police).	Body: alias, offenceCategory, station, note. Response: 201 with the case id (format ES-year-8 hex characters). 403 if not Police.
GET	/api/cases/:id	View one case the caller's role may access.	Response: 200 with the case and its event history, or 404 if not available to the caller's role. Writes a case_viewed audit event.
POST	/api/cases/:id/handoff	Move a case on: refer to Prosecution, refer to court, hand off to custody or close administratively. Allowed only for the agency holding the case.	Body: stage (the next stage) and note. Response: 200 with the updated case. 409 if the role or stage does not allow the move; 404 if the case is not visible to the role.

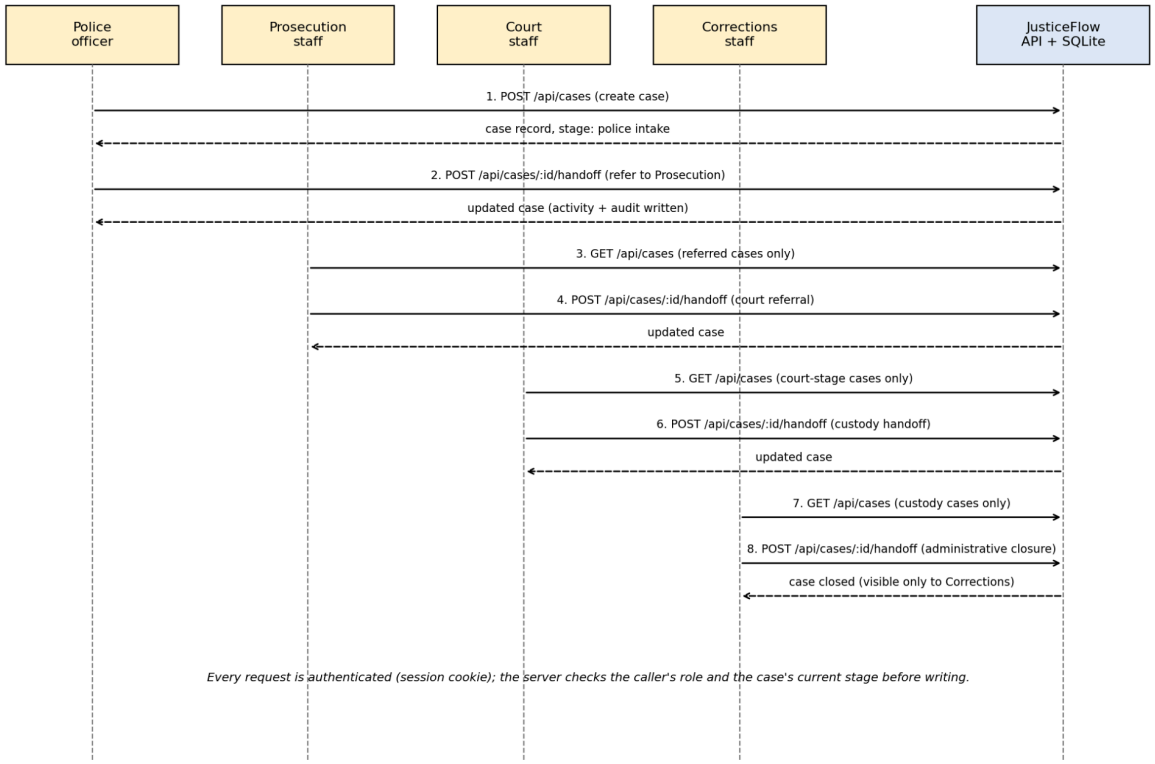
3.5 Class Diagram

Figure 3.2: Class diagram of the logical data model: User, Session, Case, CaseEvent and AuditEvent, matching the five tables in the SQLite database.



3.6 Sequence Diagram

Figure 3.3: Sequence of the main flow. Police create and refer a case, Prosecution records a court referral, the Court records a custody handoff and Corrections records the closure. Each step is an authenticated request that the server checks before writing the case and its activity and audit entries.



3.7 Data Design and Security

Data: the SQLite database has five tables: users, sessions, cases, case_events and audit_events. A case moves through five stages: Police intake, Prosecutor review, Court proceedings, Correctional custody and Closed.
Authentication: users sign in with a username and password. Passwords are salted and hashed with scrypt. Session tokens are random, only their SHA-256 hashes are stored, they expire after 8 hours, and they travel in an HttpOnly, SameSite=Strict cookie named justice_session. Changing a password ends the user's other sessions.
Authorization: the server checks the caller's role and the case's current stage on every request. Hiding a button in the interface is not relied on. Each agency sees only its current-stage records, a closed record stays visible only to the agency that closed it, and the administrator cannot view case records.
Protection: request bodies are limited to 16 KB; after 10 failed sign-ins in 15 minutes an address is blocked for a while (HTTP 429); every state-changing request must come from the same origin; inputs are checked for length and unsupported characters; and responses carry a Content-Security-Policy and other security headers. Each case change runs in a database transaction that also checks the old stage, so two users cannot move the same case at once. A case outside the caller's role returns 404, so its existence is not revealed. The server binds to the loopback address by default.

4. Implementation Details

4.1 Technology Stack and Environment

- Runtime: Node.js 22.13 or later, using the built-in node:sqlite module. No third-party packages are installed.
- Database: SQLite file at data/justice.sqlite. Set JUSTICE_DB_PATH to use another file.
- Run: npm start, then open http://127.0.0.1:4173. The server binds to the loopback address by default.
- First administrator: run npm run create-admin in an interactive terminal. It asks for a username, display name and password (at least 12 characters, hidden while typed) and refuses to create another administrator unless ALLOW_ADDITIONAL_ADMIN=true is set.
- Validation: npm run check and npm test.

4.2 Project Structure

Each role has its own page: /police, /prosecution, /court, /corrections and /admin, plus the public landing page and /login. All records are stored in data/justice.sqlite.

- server.js: HTTP server, routing, the REST API and security headers.
- db.js: opens the SQLite database and creates the tables users, sessions, cases, case_events and audit_events.
- auth.js: password hashing (scrypt), session tokens and identifiers.
- site/: the web pages and scripts (index.html, login.html, workspace.html, login.js, workspace.js, site.css, workspace.css).
- scripts/create-admin.js: the command that creates the first administrator.
- integration.test.js (in the project's test folder): the automated integration test run by npm test.
- data/justice.sqlite: the database file, created on first run.

4.3 Key Implementation Highlights

- Authentication: scrypt with a per-user salt. Random session tokens, with only their hashes stored. HttpOnly, SameSite cookies.
- Forced password change: a user with a temporary password must change it before a workspace opens.
- Stage-based visibility: GET /api/cases returns only the caller's current-stage cases. A closed case stays visible only to the agency that closed it.
- Controlled hand-overs: POST /api/cases/:id/handoff checks that the caller's role matches the case's current stage and that the requested transition is allowed.
- Audit: every database write adds case activity and audit events.
- Hardening: request size limits, login throttling, input validation and security headers.
- Separation of duties: the administrator can create accounts but cannot read case records.

From server.js, the rules that decide who may move a case and to which stage:

```
const transitions = {
  police: { from: "Police intake", to: ["Prosecutor review"] },
  prosecution: { from: "Prosecutor review", to: ["Court proceedings", "Closed"] },
  court: { from: "Court proceedings", to: ["Correctional custody", "Closed"] },
  corrections: { from: "Correctional custody", to: ["Closed"] }
};
```

4.4 Demonstration Scenario

Use fictional test data only.

1. Start the server with npm start and create the administrator with npm run create-admin.
2. Sign in at /login as the administrator and create one account each for Police, Prosecution, Court and Corrections.
3. Sign in as each new user and change the temporary password.
4. Police: create a case and refer it to Prosecution.
5. Prosecution: open the referred case and record a court referral.
6. Court: open the case and record a custody handoff.
7. Corrections: view the custody case and record an administrative closure.
8. Show that each agency sees only its current-stage cases and that the administrator cannot view case records.

Figure 4.1: Public landing page with the workflow overview, constitutional safeguards and staff sign-in link.

JUSTICE ADMINISTRATION · ESWATINI

A connected journey. Accountable at every step.

A shared case workflow concept for authorized justice staff — from police intake and prosecution review to court proceedings and correctional services.

Authorized staff sign in → Explore the workflow ↓

✓ Role-based staff access · Persistent case records · Logged handoffs

ⓘ This starter is not an official government service and is not approved for real case data.

A CLEARER HANDOFF

One case. Separate responsibilities.

Each team signs in to its own workspace. Updates are attributed to a user and time-stamped; the system does not decide legal outcomes.

01 → 02 → 03 → 04

Police intake

Create an initial case record and make an administrative referral.

Prosecution review

Receive the handoff and record the next administrative action.

Court proceedings

Track a proceeding and record an authorized onward handoff.

Correctional services

Record a custody handoff and its administrative completion.

RIGHTS MUST COME FIRST

Figure 4.2: The administrator signs in at /login.

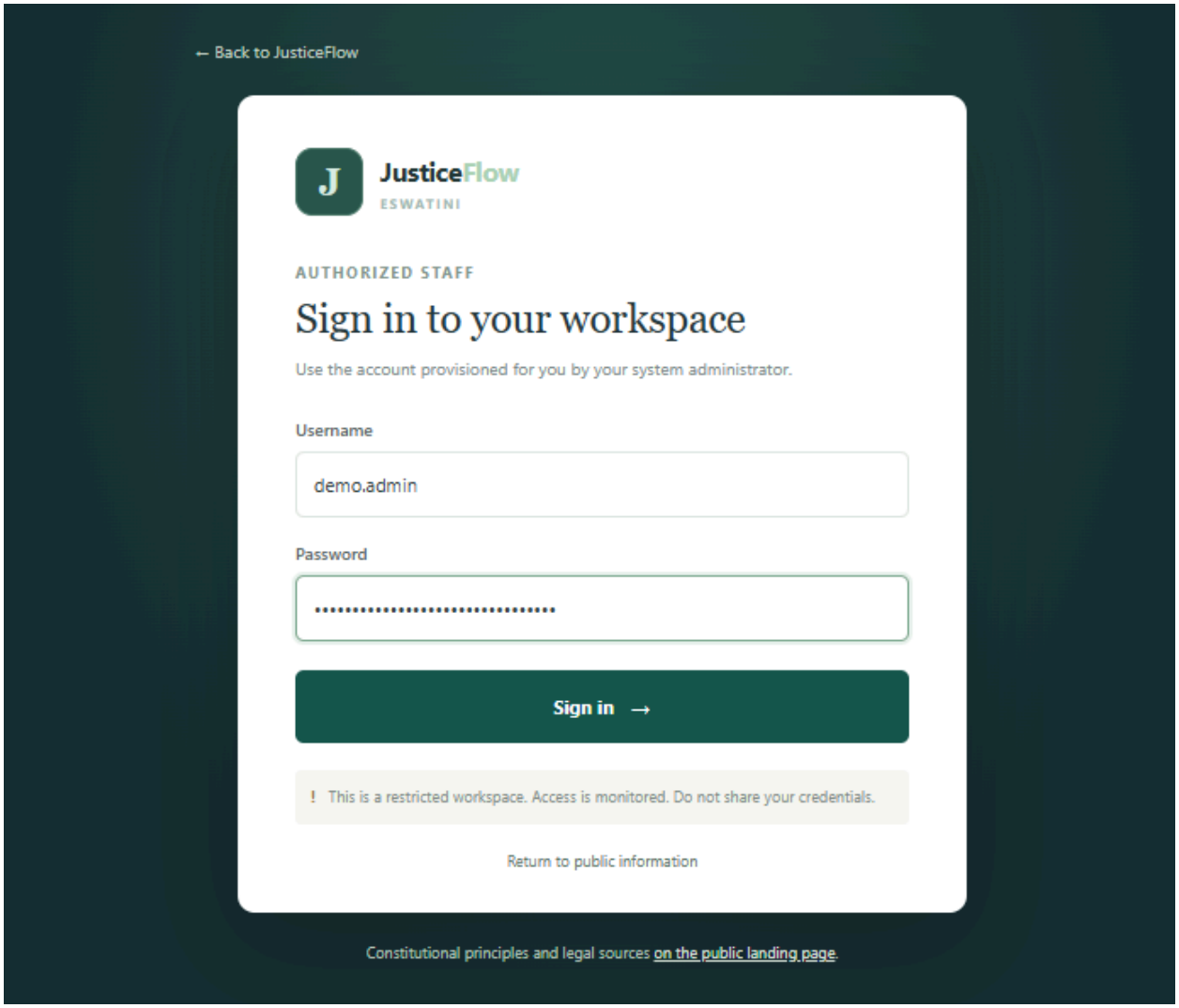


Figure 4.3: Administrator workspace: the staff account form. No case register is shown.

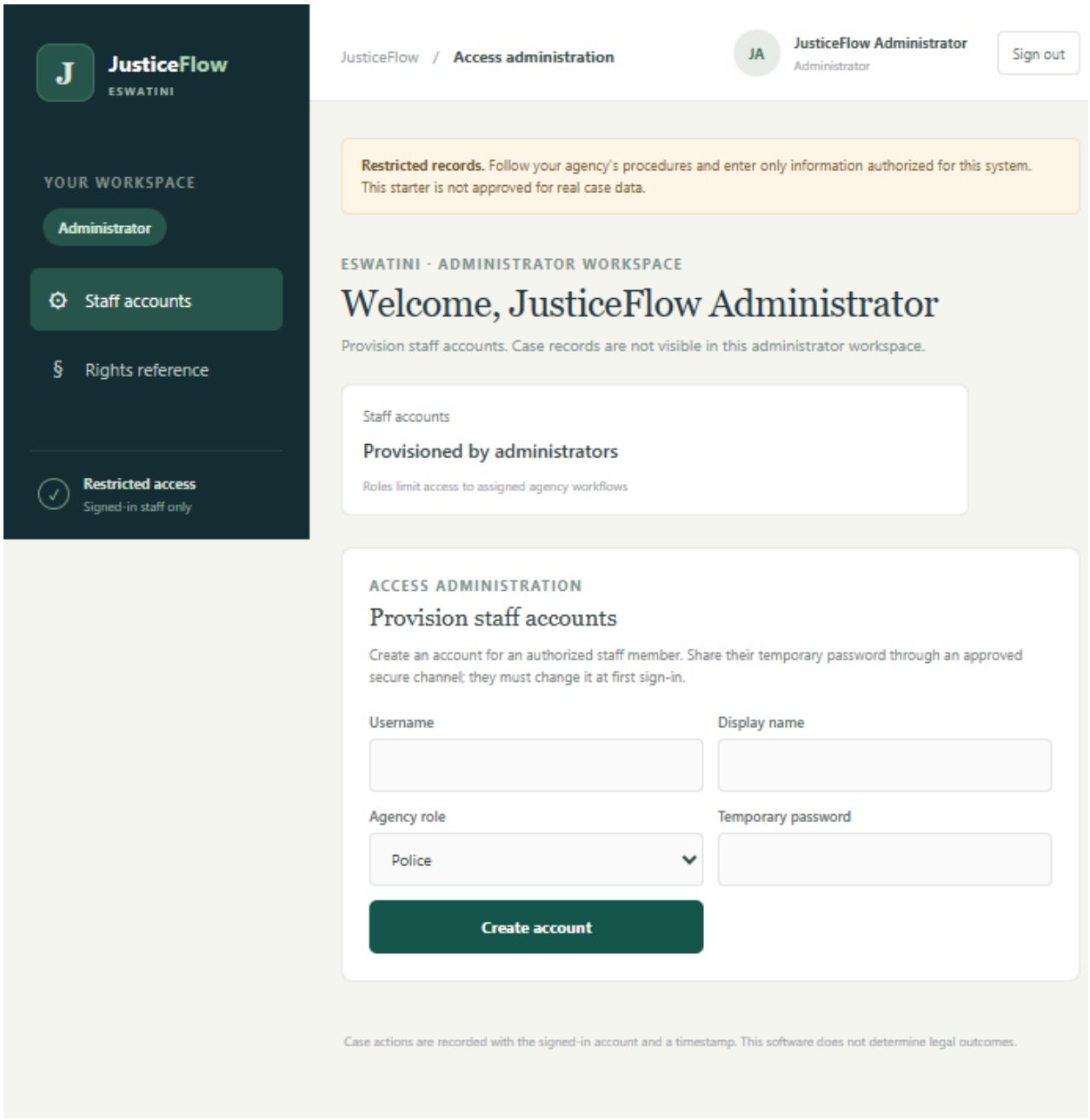


Figure 4.4: A Prosecution account created by the administrator. Court and Corrections accounts are created the same way.

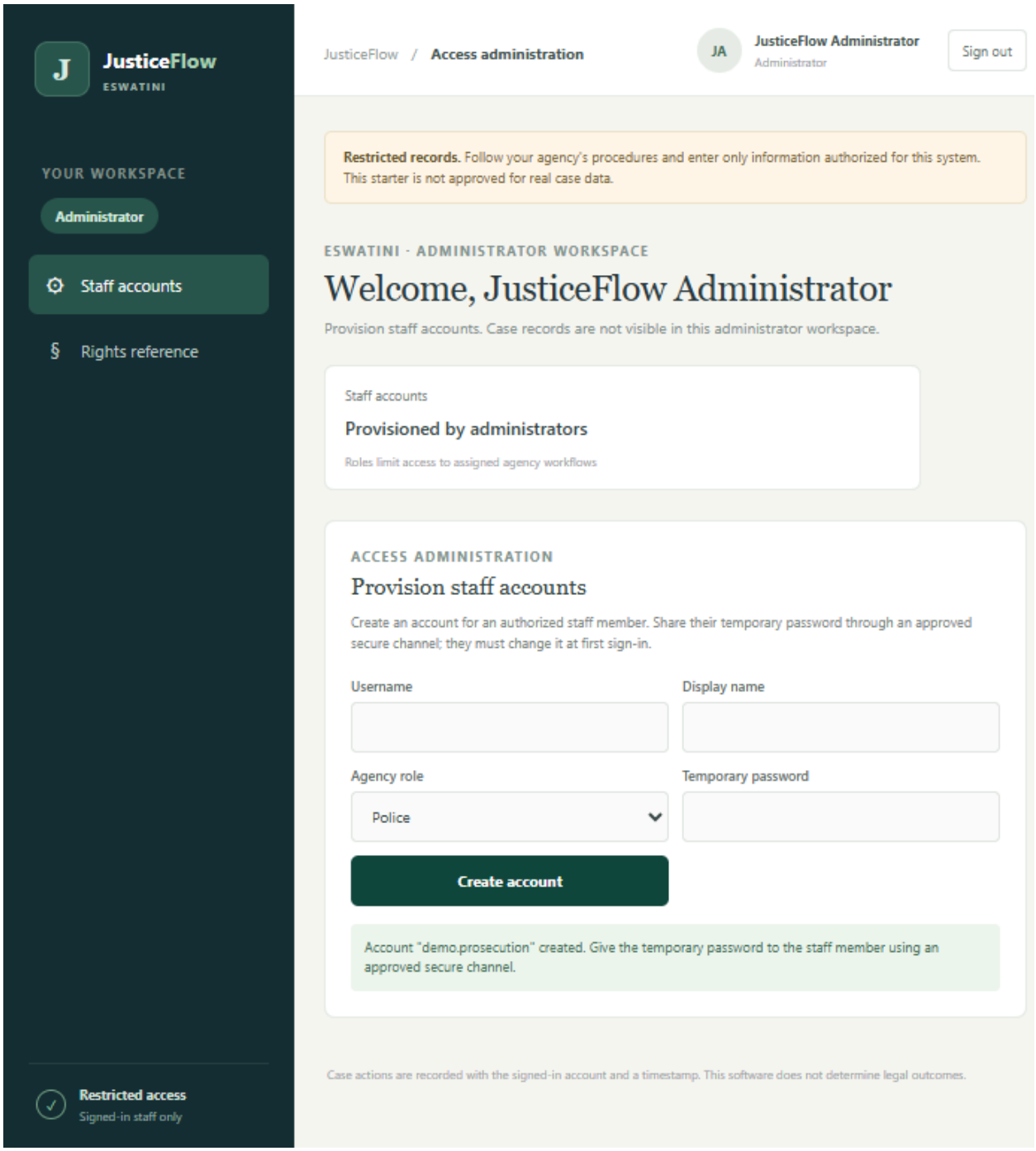


Figure 4.5: Police staff sign in with the temporary password.

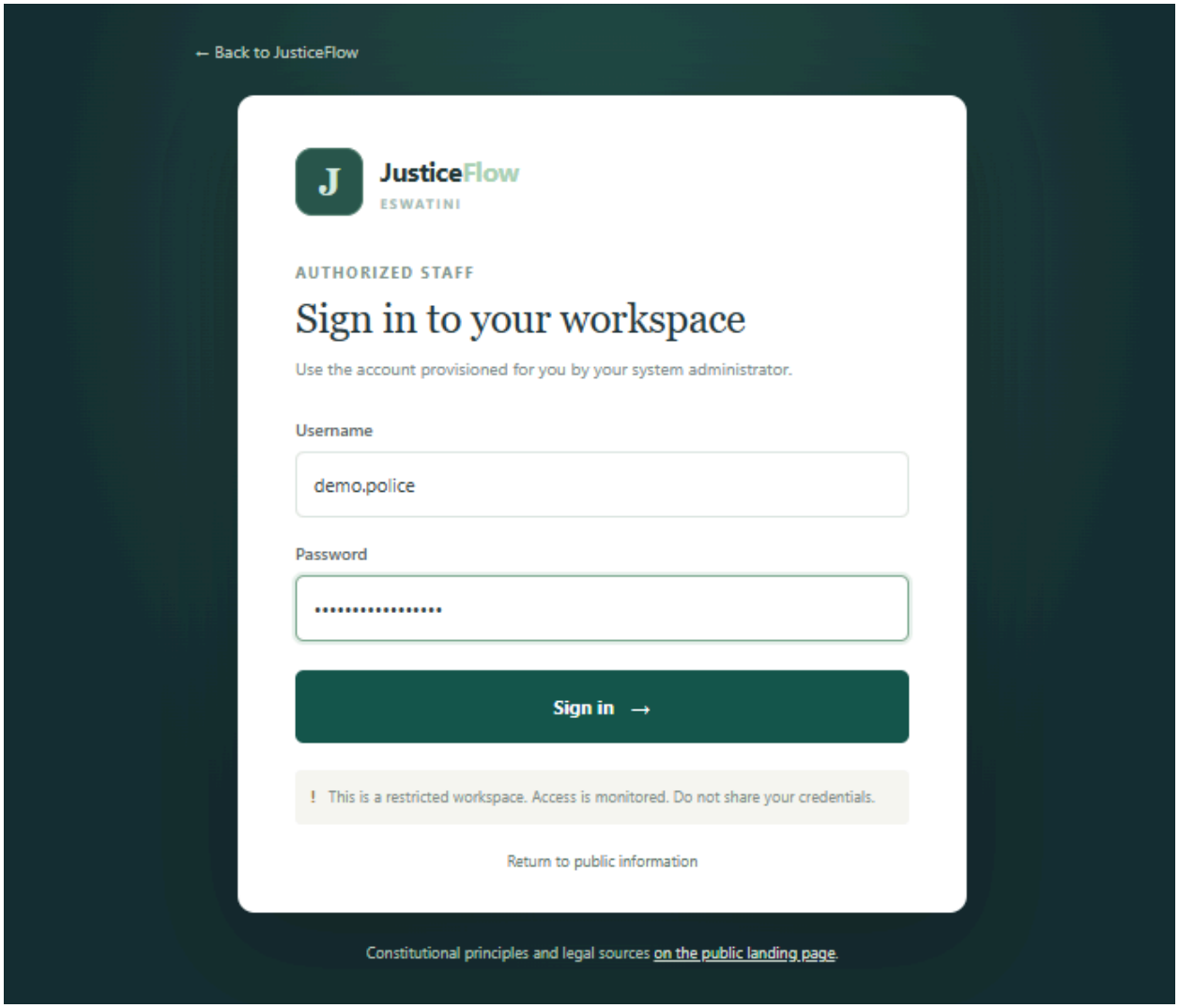


Figure 4.6: The required password change before the workspace opens.

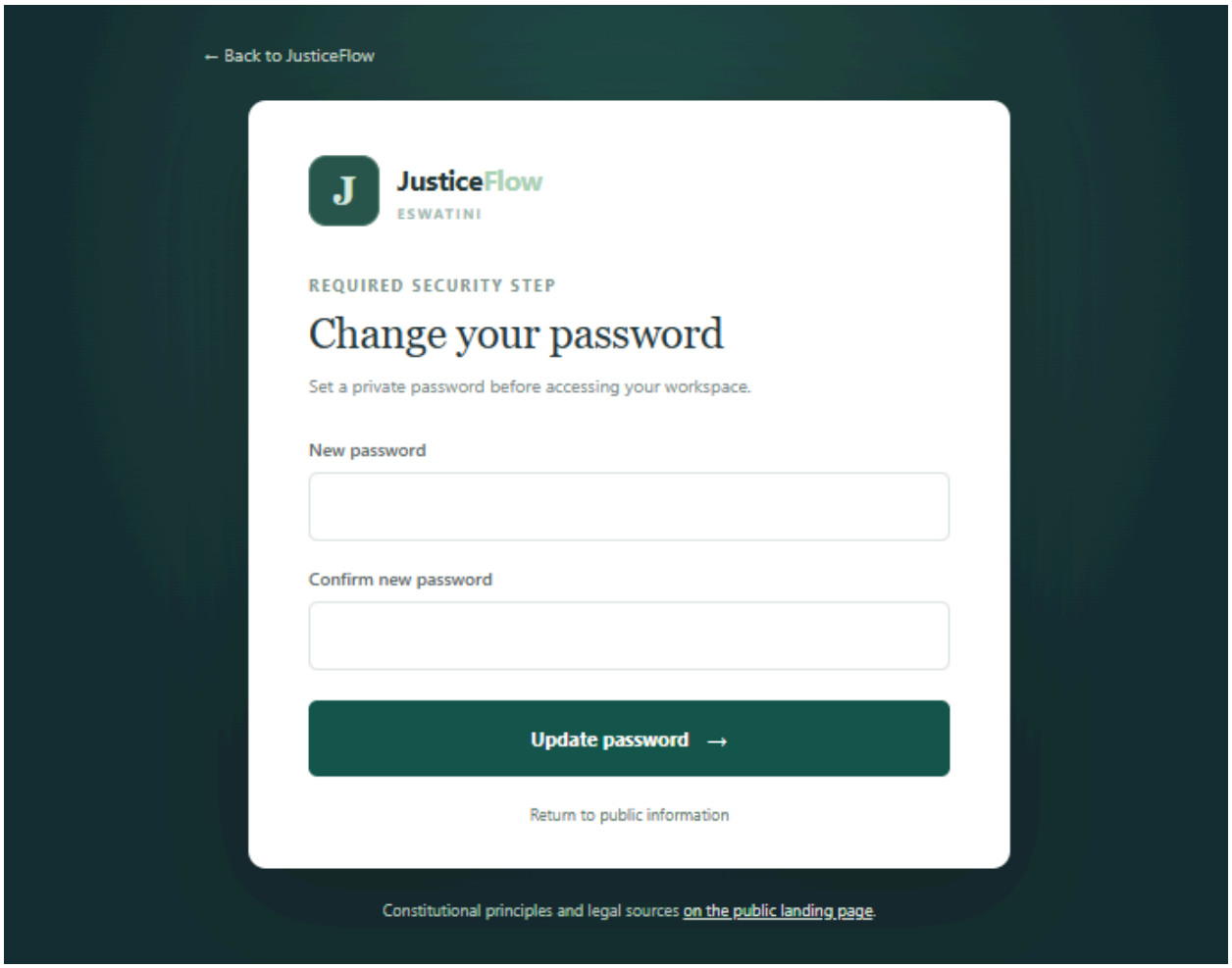


Figure 4.7: Police workspace with an empty case register.

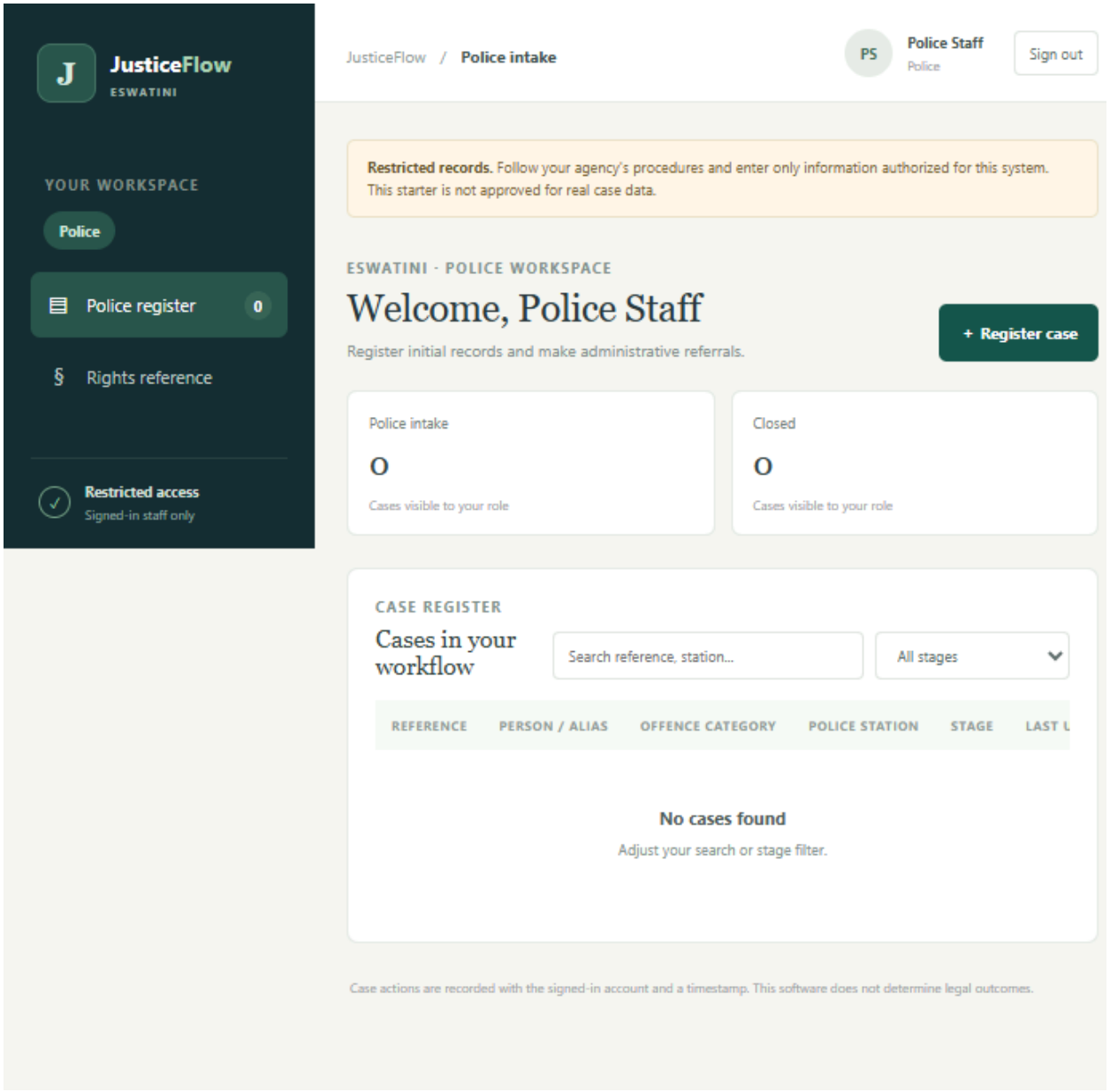


Figure 4.8: Police register a fictional case.

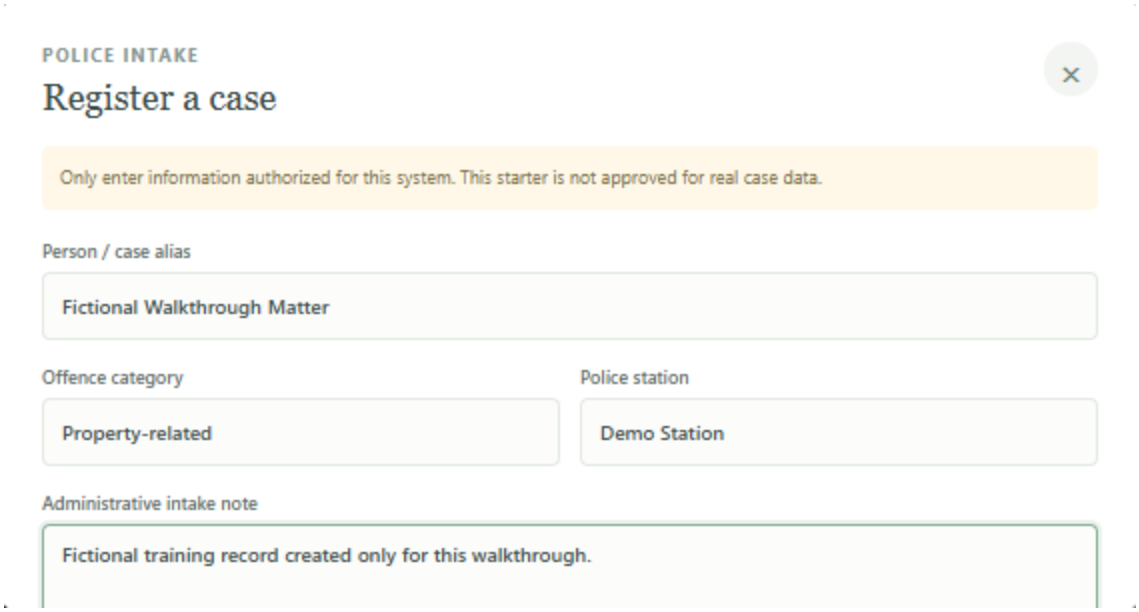


Figure 4.9: Case ES-2026-16625496 created in the Police intake stage.

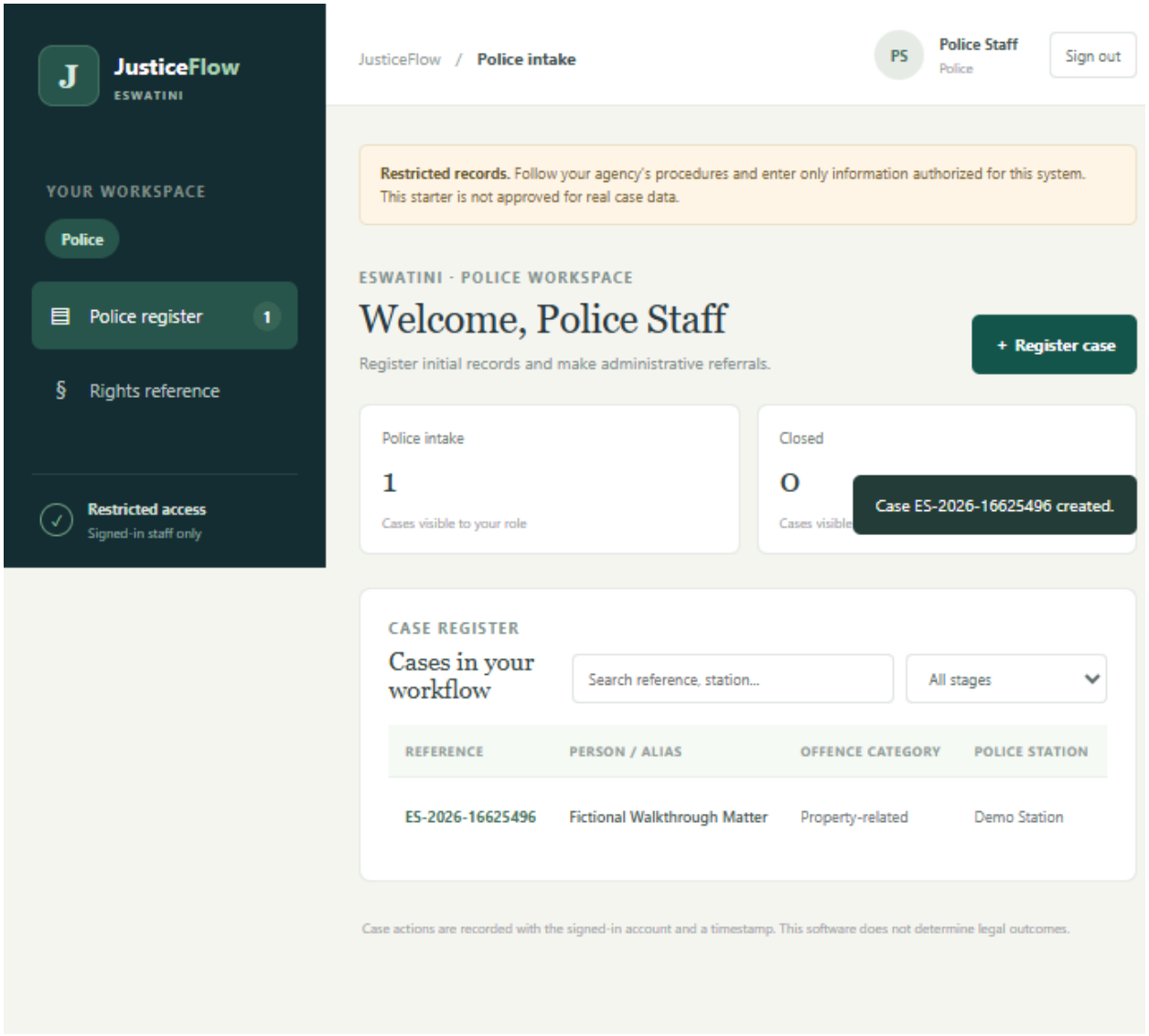


Figure 4.10: Police record the handoff to Prosecutor review.

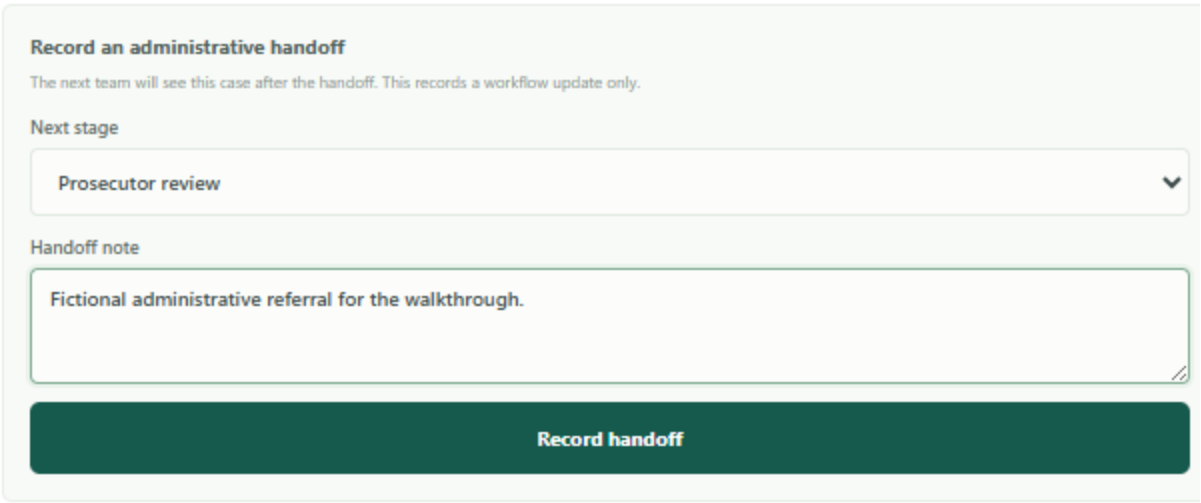


Figure 4.11: After the handoff, the Police register shows 0 cases.

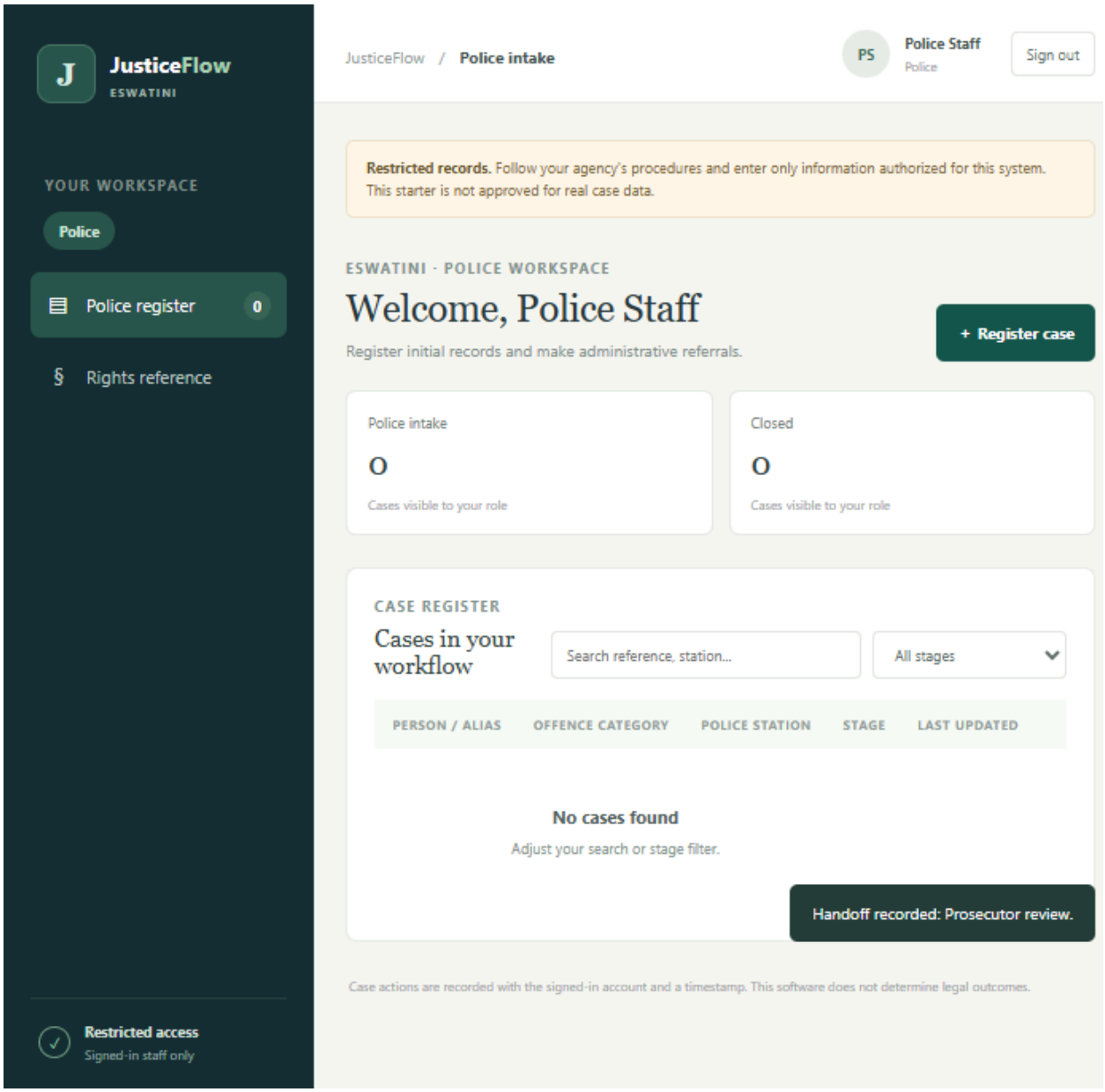


Figure 4.12: The Prosecution workspace shows the referred case.

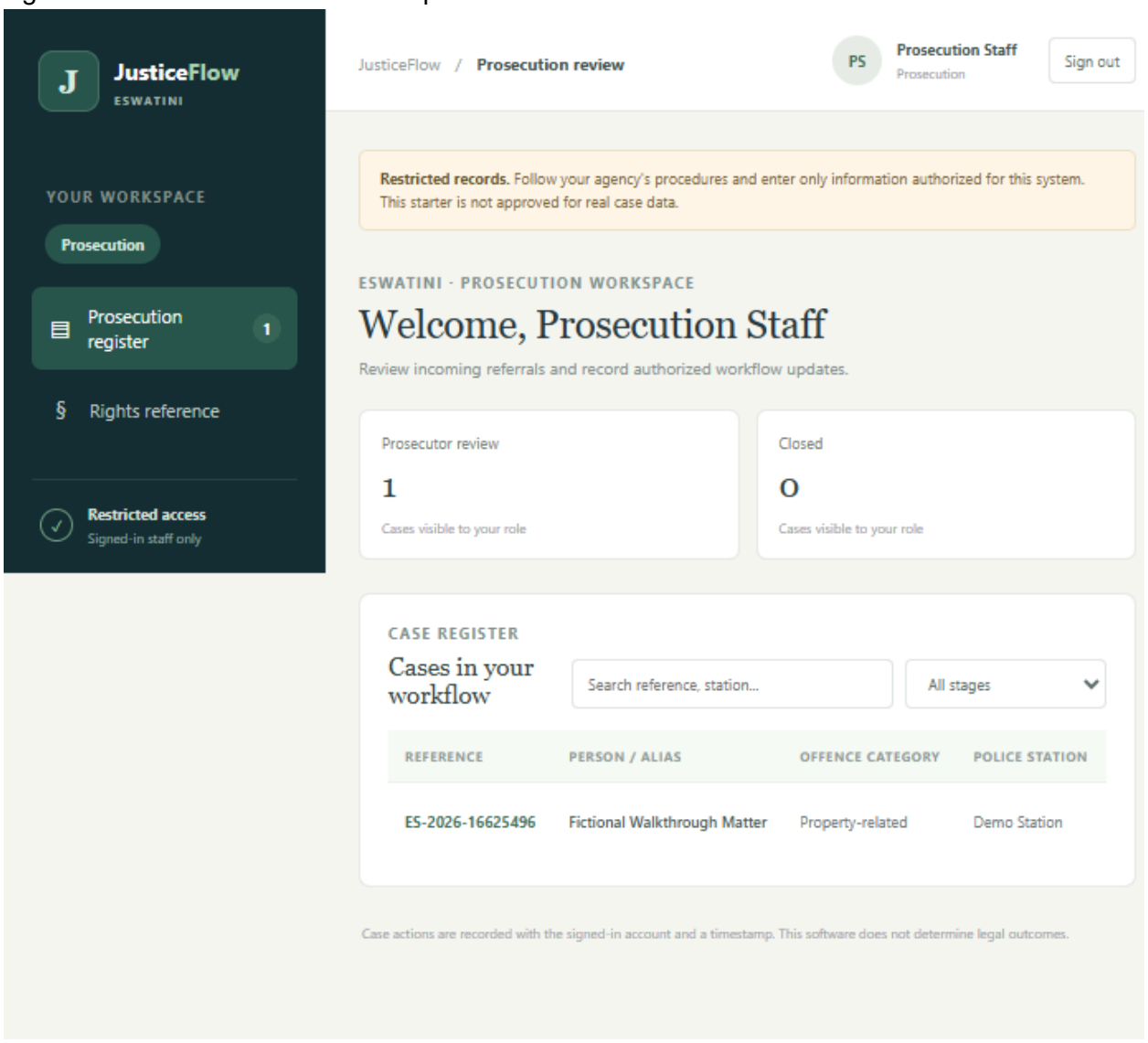


Figure 4.13: Prosecution records the handoff to Court proceedings.

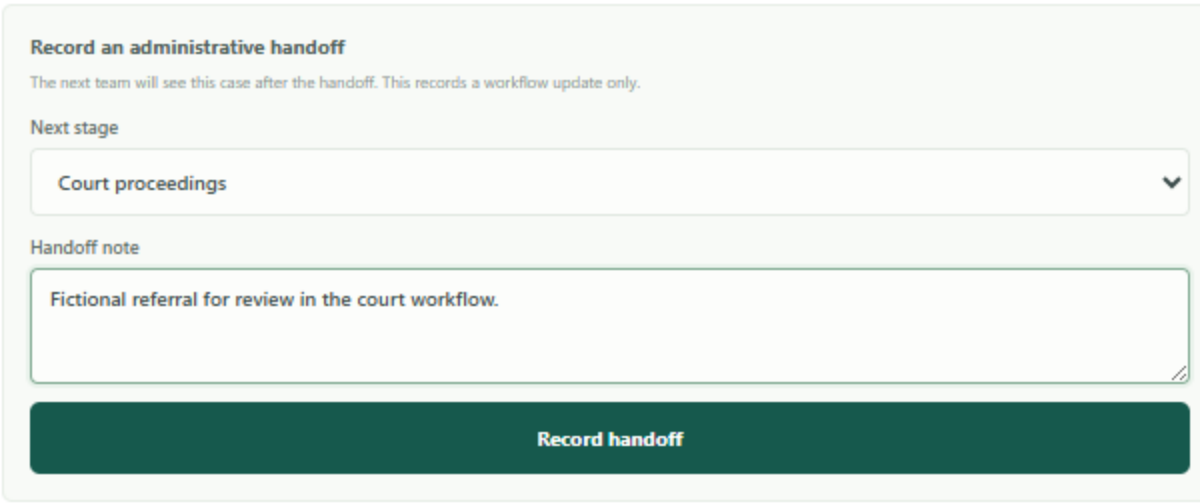


Figure 4.14: After the handoff, the Prosecution register shows 0 cases.

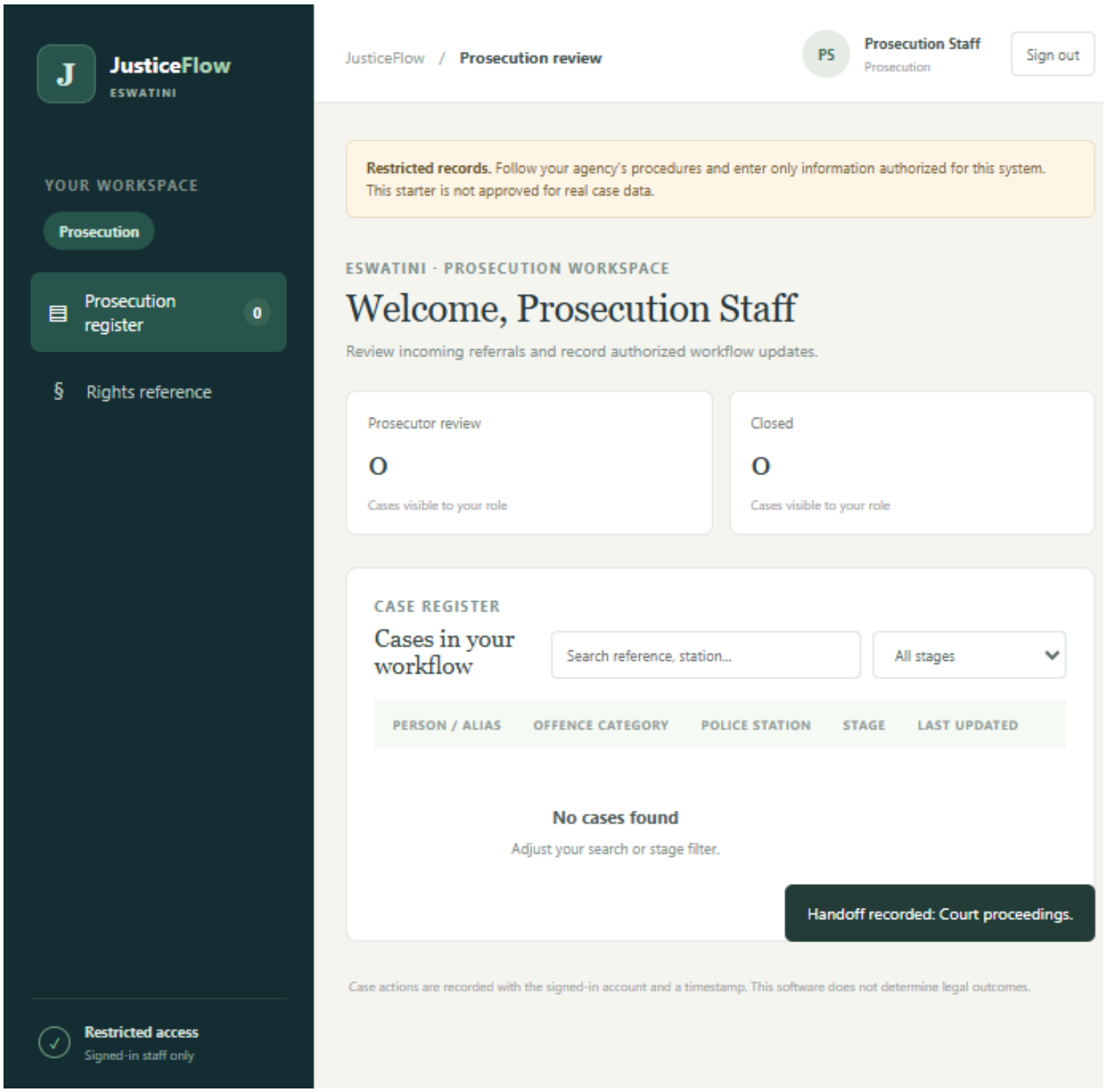


Figure 4.15: The Court workspace shows the case.

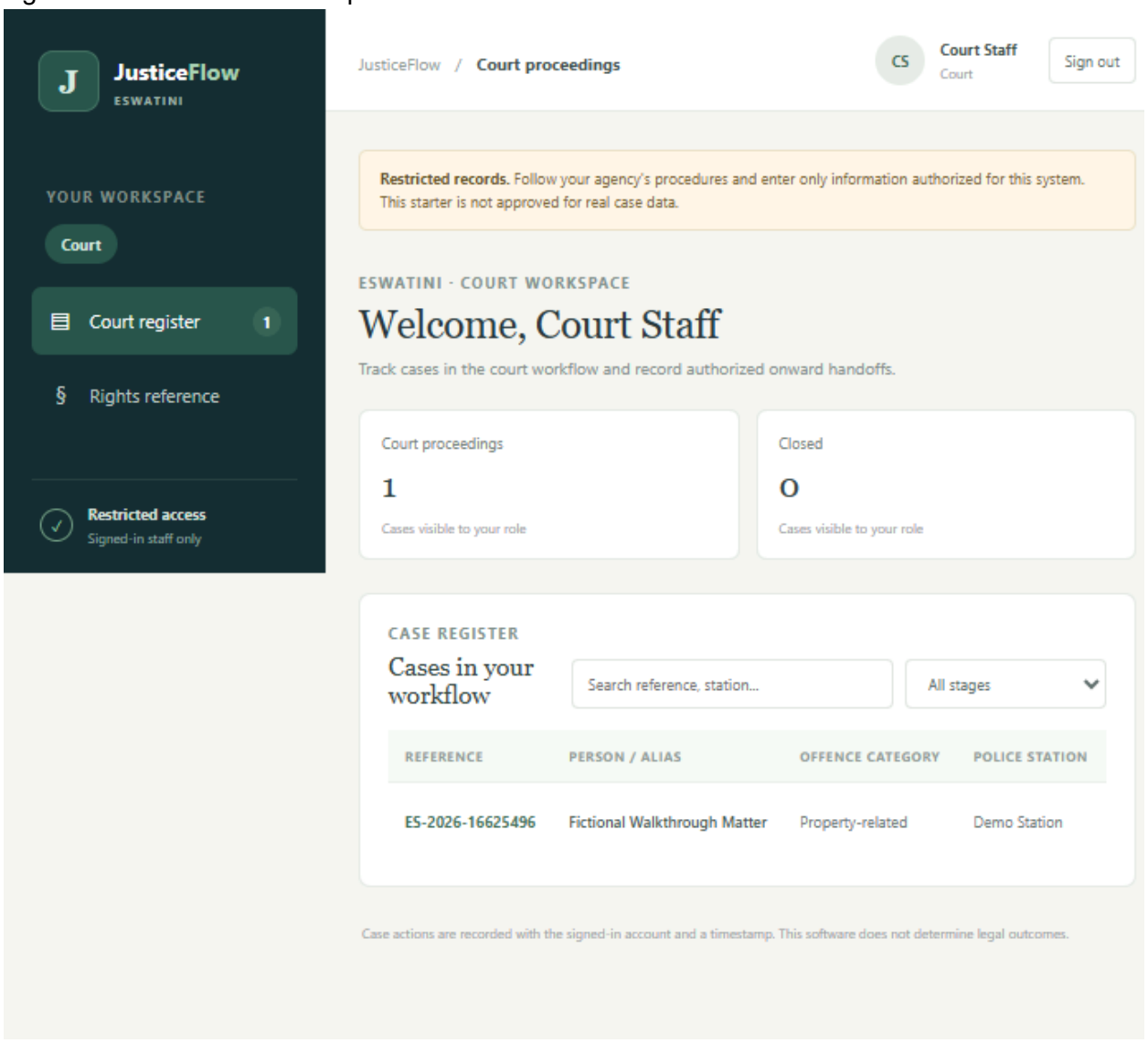


Figure 4.16: Court record the handoff to Correctional custody.

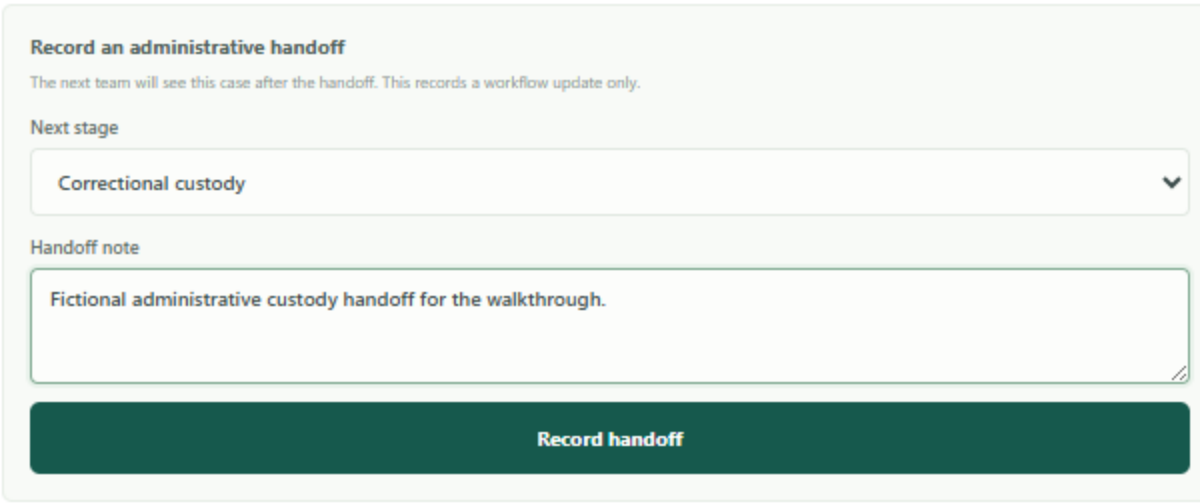


Figure 4.17: After the handoff, the Court register shows 0 cases.

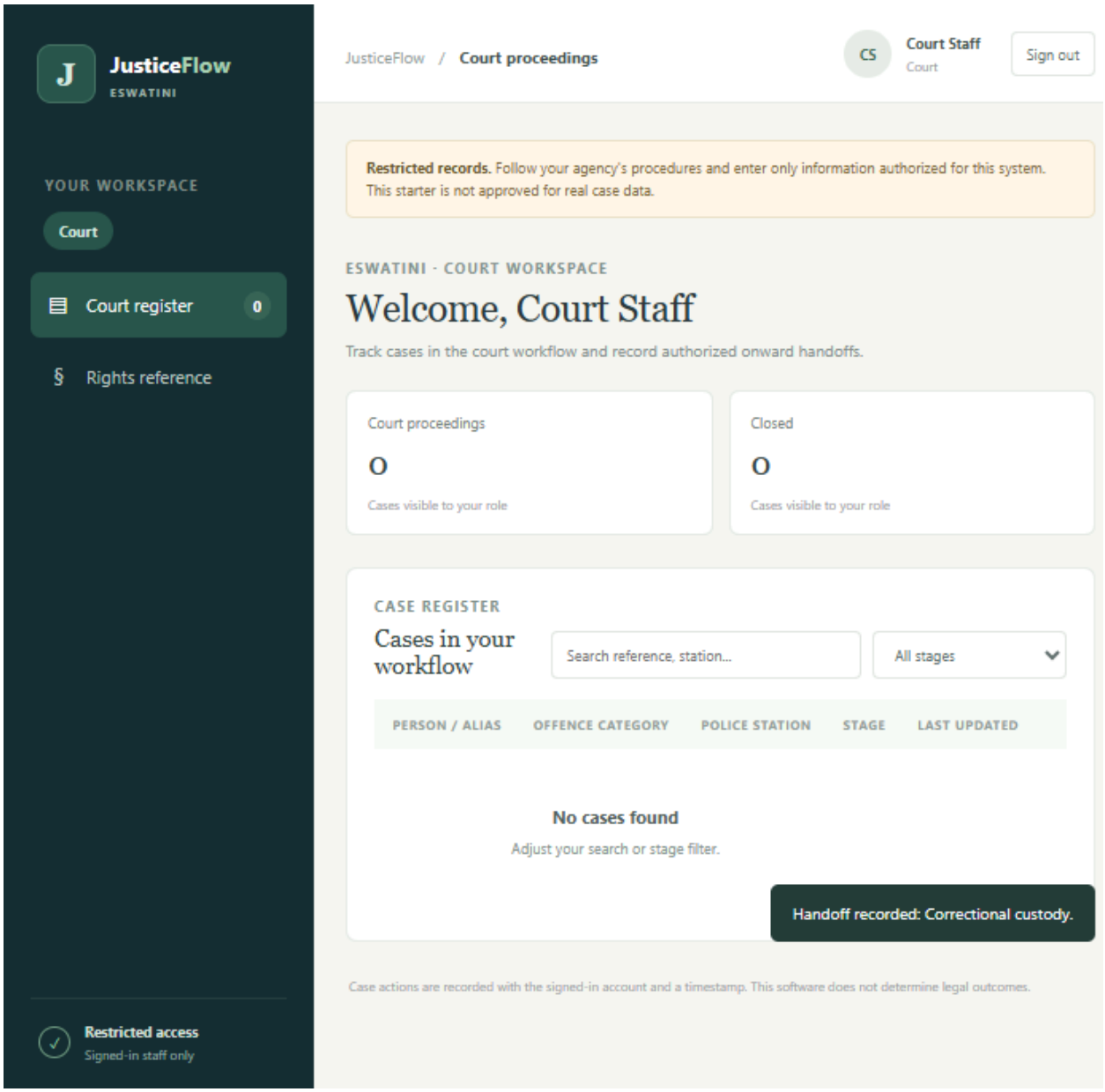


Figure 4.18: The Corrections workspace shows the custody case.

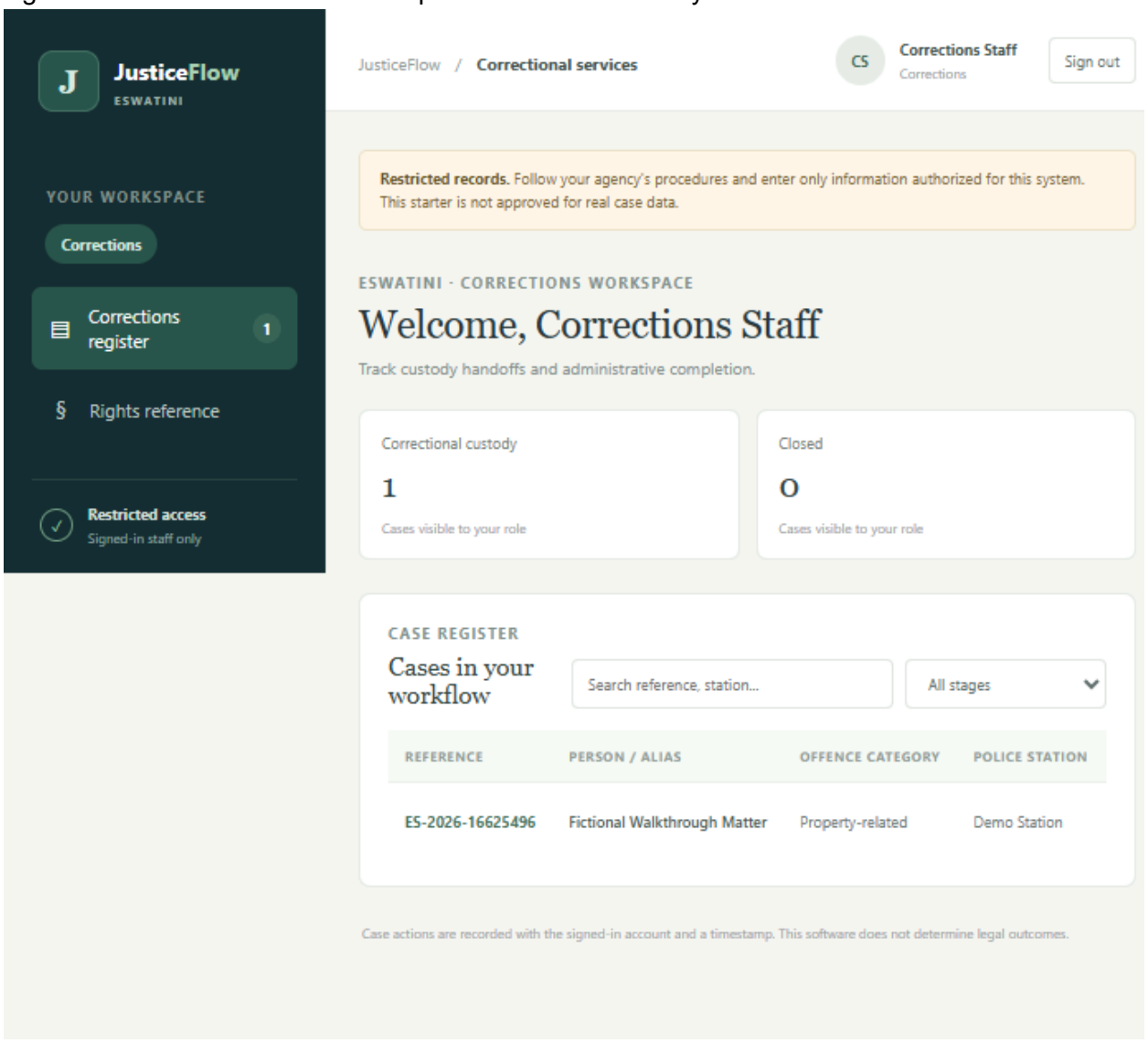


Figure 4.19: Corrections close the case.

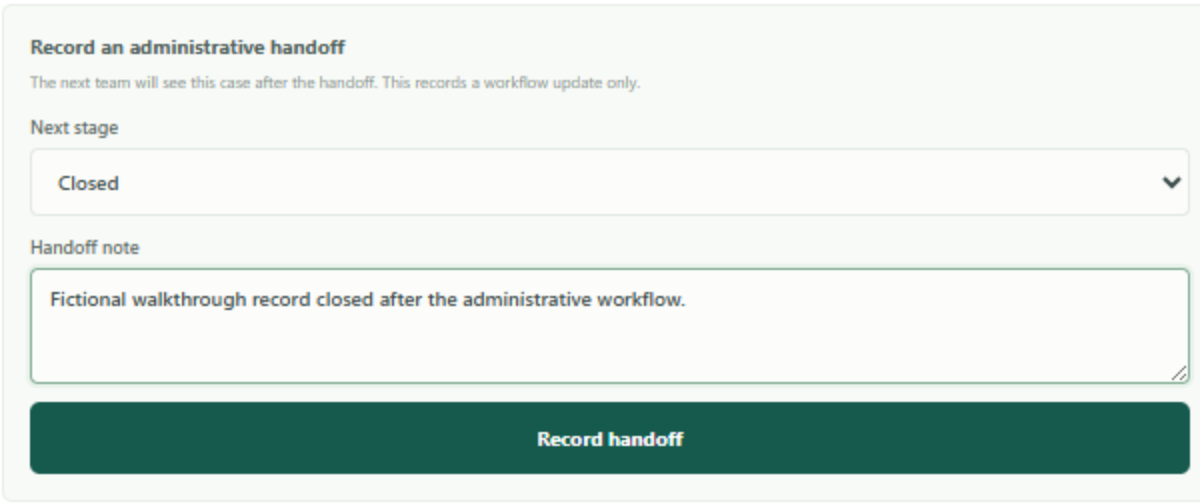
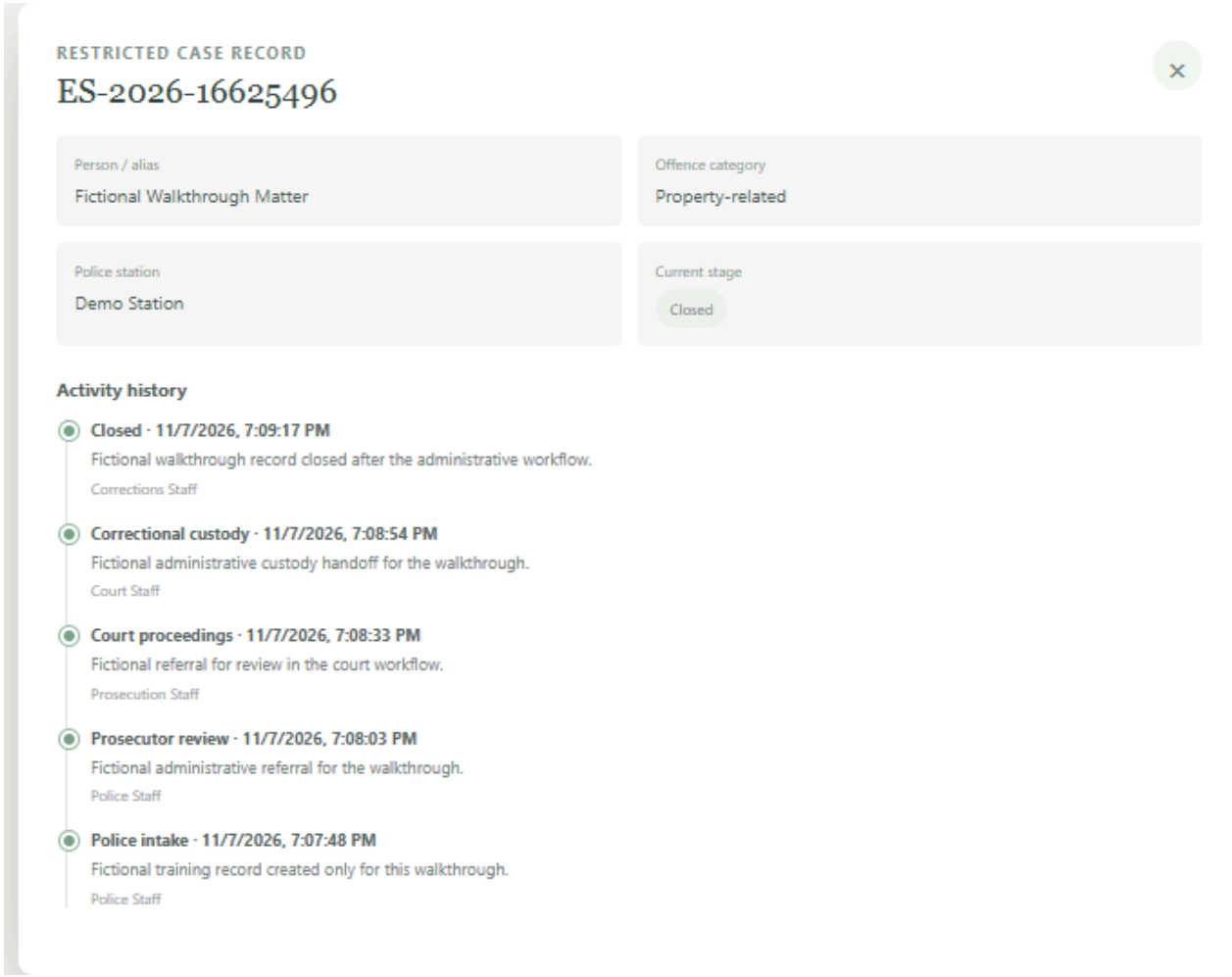


Figure 4.20: The closed case with its full activity history, each entry attributed to the staff member who made it.



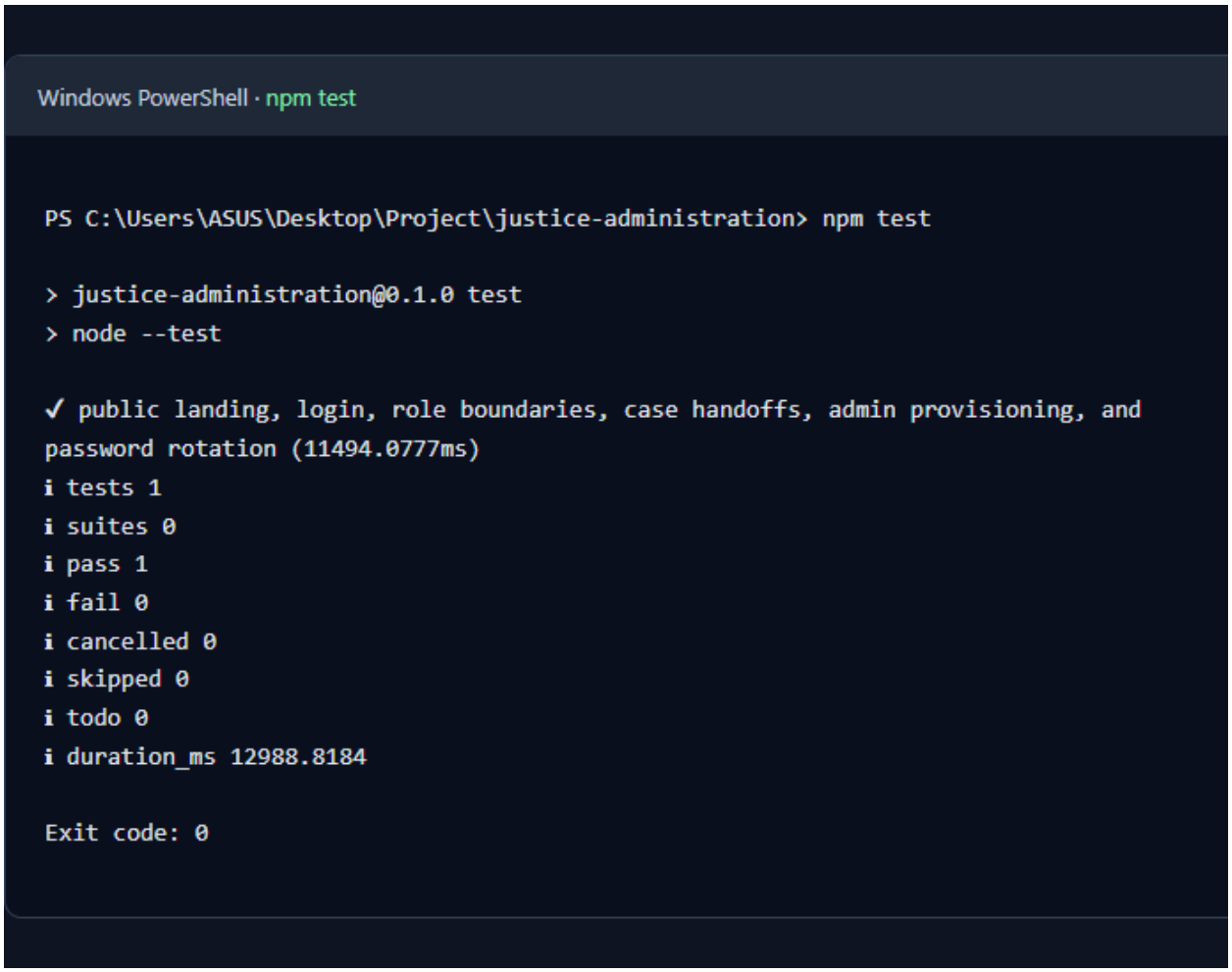
4.5 Testing

npm run check validates the syntax of every source file. npm test runs an integration test that starts the server on a free port with a temporary database and five seeded accounts (admin, police, prosecution, court, corrections). It checks that: the landing page shows the rights overview and the security headers; an anonymous visit to /police redirects to /login; a sign-in request without a same-origin header is refused (403); the administrator cannot read cases (403); Prosecution cannot create a case (403); a case created by Police moves through Prosecutor review, Court proceedings and Correctional custody to Closed, and after each hand-off the previous agency gets 404 for it; skipping a stage is refused (409); the closed case stays visible only to Corrections, with five events in its history; an account created by the administrator gets 428 until its password is changed, after which the old password fails (401) and the new one works; and a session is rejected after sign-out (401). The test passes (1 test, 0 failures). Manual Postman requests can repeat the same cases against the running server.

Figure 4.21: Result of npm run check (exit code 0).



Figure 4.22: Result of npm test: 1 test passed, 0 failed.



4.6 GitHub Links

#	Member	GitHub profile URL
1	Ndlovu Dumisa	https://github.com/chrvso
2	Mkhatshwa Fisokuhle	https://github.com/mkhatshwafisokuhle22-sketch
3	Ginindza Nkosephayo	https://github.com/Phayo121
4	Khumalo Owenkhosi	https://github.com/khumaloowenkhosi
5	Magagula Sibonelo	https://github.com/Sibonelo623

Project repository: https://github.com/Phayo121/justiceflow

4.7 Challenges and Limitations

- Single deployment: all agencies are modules of one Node.js server and one database. They are not independently deployed systems. In a real rollout each agency would run its own system and exchange cases through agreed APIs, and the role and stage rules would become contracts between those systems.
- Not production-ready: there is no built-in HTTPS (production needs a reverse proxy with HTTPS), and no multi-factor sign-in or account recovery. The audit log is not tamper-proof and SQLite suits a single node.
- No real data: only fictional test data may be entered.
- Decisions are not automated: the system records hand-overs. Arrest, charge, guilt, bail, sentence and release remain decisions for the responsible officials.
- Legal validation: roles and hand-offs must be validated by the competent Eswatini authorities and counsel before any real use.

5. Conclusion

JusticeFlow shows how a justice case can move between Police, Prosecution, Court and Corrections through one authenticated system. The prototype meets the aims set in Section 1: it describes the process and its stakeholders, provides a REST-style API with role and stage checks, stores records persistently in SQLite, and keeps activity and audit records for every case write.

Its main limits are that the agencies are modules of a single deployment rather than separate systems, and that it is a prototype that must not hold real case data. Future work includes giving each agency its own service with API contracts between them, adding HTTPS, multi-factor sign-in and account recovery, storing the audit log in a tamper-evident way, and validating roles and hand-offs with the competent authorities.

6. References

1. Constitution of the Kingdom of Swaziland Act, 2005 (now Eswatini), Chapter III, Fundamental Rights and Freedoms. https://www.constituteproject.org/constitution/Swaziland_2005
2. Fielding, R.T. (2000) Architectural Styles and the Design of Network-based Software Architectures. PhD dissertation, University of California, Irvine.
3. JusticeFlow project README and source code (2026) GitHub repository. https://github.com/Phayo121/justiceflow
4. Node.js documentation: Crypto (scrypt). https://nodejs.org/api/crypto.html
5. Node.js documentation: SQLite (node:sqlite). https://nodejs.org/api/sqlite.html
6. Nxumalo, M.A. (2026) CSC411 Integrative Programming Technologies: Assignment 1, Mini Project. University of Eswatini, Department of Computer Science.
7. SQLite documentation. https://www.sqlite.org/docs.html